Quantized Transformer Inference on CPUs and GPUs

1. Introduction

1.1 Background

Transformer based models, such as BERT and GPT, have shown total dominance in the NLP scene. However, these models are famed for being resource hogs. The self-attention mechanism introduces a complexity of O(n^2) proportional to the sequence length. In prod environments, this architecture translates to high latency, high memory consumption, and strains budgets. Efficiently serving these models is now a necessity.

1.2 Problem Context

While the AI industry races towards bigger models, the reality is that the majority of environments are STILL heavily constrained by heterogeneous, and legacy hardware. Production environments rarely consist only of the latest and greatest accelerators. These may be CPUs, older, and newer GPUS.

The core problem in optimizing for these environments lies in the Quantization-Efficiency Tradeoff. PTQ (Post-Training Quantization) to INT8 theoretically decreases the memory usage and increases the inference speed. However, actual performance is volatile. They are bottlenecked by hardware-specific parallelization capabilities and the presence/absence of Tensor Cores.

1.3 Research Gap: The Hardware-Aware Quantization Void

While literature on model quantization is vast, a gap exists in evaluating how it behaves across diverse silicon generations. Specifically:

1. Isolated Benchmarking: Most studies evaluate GPU inference in a void and do not benchmark it directly against high optimized CPU environments (which are quite competent for batch_size = 1 inference).
2. Architectural Discrepancies: There is insufficient empirical data regarding the performance cliff between newer architectures that natively support INT8 matrix multiplication and older ones that emulate it.
3. Scaling Ambiguity: Existing literature sometimes ignores the play between batch size and INT8 memory bandwidth across different architectures.

1.4 Research Aim

The primary goal of this research is to design and create a reproducible benchmarking methodology to test the effects of INT8 PTQ on transformer inference across heterogeneous hardware platforms. We aim to quantify the exact tradeoffs between latency, throughput, memory util, and task accuracy.

1.5 Research Questions

To achieve this aim, this study seeks to answer the following:

RQ1: How does the underlying hardware architecture (CPU vectorization vs Pascal DP4A vs Turing Tensor Cores) dictate the actual latency decrease achieved by the INT8 PTQ?

RQ2: At what batch-size threshold does memory bandwidth saturate and how does this saturation point shift across the different environments?

RQ3: To what extent does INT8 quantization degrade performance on the GLUE benchmark, and is this degradation justified by the corresponding gains?

2. Literature Review

2.1 Transformer Bottleneck

The “Attention is All You Need” architecture established the self-attention mechanism as the standard for NLP. While revolutionary for capturing semantic context, it is inherently biased against memory-constrained environments, especially when deploying large-scale models like BERT-base. Current optimizations focus on PDC strategies to tackle these costs, attempting to restructure how these massive matrix multiplications interact with cloud hardware.

2.2 Post-Training Quantization Theory

Quantization is the process of mapping high-precision values to low-precision values (FP32 to INT8 here). PTQ is heavily favored in industry applications because it eliminates the cost of retraining the model.

- Memory Efficiency: Reducing precision from 32 to 8 bits cuts the model size by approximately 75%, which is necessary for memory-constrained scenarios.
- Computation Principles: Low-precision arithmetic significantly increases the THEORETICAL output by utilizing specialized hardware instructions.
- Precision Tradeoff: The mapping of FP32 distributions into 256 integers introduces quantization noise, and results in penalizing the model accuracy on strict benchmarks like GLUE and SQuAD.

2.3 Hardware-Aware Inference

A review of current frameworks (such as ONNX Runtime and DeepSpeed) reveals that software abstraction is only half the battle. Hardware dictates

the final performance. Newer GPUs feature dedicated INT8 Tensor Cores designed specifically for these workloads while older GPUs often lack them and yield diminished returns. Furthermore, while GPUs excel at batch processing, a highly optimized CPU scenario could achieve lower latency for sequential streams (batch_size = 1).

2.4 Identified Research Gap

A study of the baseline literature shows a Cross-Platform Benchmarking gap. While the theoretical benefits are well documented, there is no standardized, comparative framework that maps out exactly how work partitioning interacts with specific hardware to produce a performance gain or loss.

3. Proposed Methodology

3.1 Experimental Design and System Architecture

To ensure rigorous testing across a set of hardware, this study utilizes the ONNX Runtime as the unified execution engine. The benchmarking cluster consists of three distinct nodes available on the cloud:

- Node 1: Kaggle CPU (Baseline for sequential/low-batch workloads)
- Node 2: NVIDIA P100 (Representing legacy/low-tier cloud GPU acceleration)
- Node 3: NVIDIA T4 (Representing modern INT8-optimized architectures)

The testing models include BERT-base and DistilBERT, selected for their use in production pipelines.

3.2 Model Preparation and Quantization Logic

1. Baseline Export: The pre-trained PyTorch/HF models are exported to the ONNX format for a baseline

2. Quantization: We apply INT8 PTQ using the ONNX Runtime quantization suite, targeting MatMul and Attention nodes, where the bottleneck on computation is worst.
3. Data Pipeline: We utilize subsets of the GLUE and SQuAD datasets(10k-20k samples) to check inference performance without introducing extra data-loading I/O overhead.

3.3 Execution and Scaling Behavior Analysis

The core of the methodology involves stress-testing the models under varying computational loads. We run inference across a set of batch size (N=1,8,16,32,64).

We also evaluate parallel efficiency by measuring multi-threading limits on the CPU and multi-gpu scaling behaviors (T4x2 on Kaggle).

3.4 Evaluation Metrics and System Validation

To prevent subjective interpretation of the results, the system is evaluated on the following metrics:

1. Predictive Integrity: We validate the quantized model's accuracy against the FP32 baseline using GLUE with an acceptable reduction of 1-2%.
2. Throughput (P): Measured as the volume of data processed per unit of time

$$P = N_{Tokens}/T_{Total}$$

3. Speedup Ratio(S): The primary metric for evaluation of success of the quantization on any given node, comparing baseline FP32 latency (L_{32}) against the quantized latency (L_{INT8}):

$$S = L_{FP32}/L_{INT8}$$

4. Memory Optimization: Peak VRAM/RAM consumption (MB) is recorded via either native hardware profiles (nvidia-smi) or by the resource charts provided by Kaggle during inference forward pass to see if the theoretical 75% memory reduction is true in practice.

3.5 Comparative Analysis

The final phase entails a rigorous architectural deepdive. By plotting latency against batch size and throughput scaling, we will explicitly identify the performance cliffs and memory bandwidth saturation points, proving why the identical software optimizations behave radically differently depending on the silicon executing the code.

References

- Aminabadi, R. Y., Rajbhandari, S., Zhang, M., Li, C., Yuan, Z., Zhe, W., ... & Rasley, J. (2022). DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 4171-4186.
- ONNX Runtime developers. (2021). ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator. Available at: https://onnxruntime.ai/
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2018). GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 353-355.

- Wu, H., Judd, P., Zhang, X., Isaev, M., & Micikevicius, P. (2020). Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation. *arXiv preprint arXiv:2004.09602*.